MemSolve: From Sequential Memory Updates to Global Equilibrium

Zhaoyang Wang
zhaoy@uir.edu.cn

Working draft — October 14, 2026

Abstract

Error-correcting associative memories update their state using the mismatch between stored predictions and incoming content. For tasks with access to the complete input, this raises a question: can all associations determine one joint memory without an ordered sequence of updates? We introduce MemSolve, a bidirectional token mixer that evaluates such a memory at equilibrium. An input-dependent soft frame aggregates contextual associations, and a learned accretive core defines a uniquely solvable correction system. The same frame reads the memory back to tokens, combining a content branch with a low-rank correction. This construction gives a contractive token map for each realized frame and permits exact spectral analysis in rank space. Although the core parameters are shared across inputs, both the frame and the memory state remain input-dependent. A compact implementation avoids materializing the frame and has linear cost in sequence length at fixed rank. On ImageNet-1K, MemSolve-ViT-S reaches 80.95% Top-1 accuracy with 19.88M parameters after 300 epochs at 224×224 resolution. A matched ImageNet-100 study further finds that learning the compact core improves accuracy over the content-complement variant.

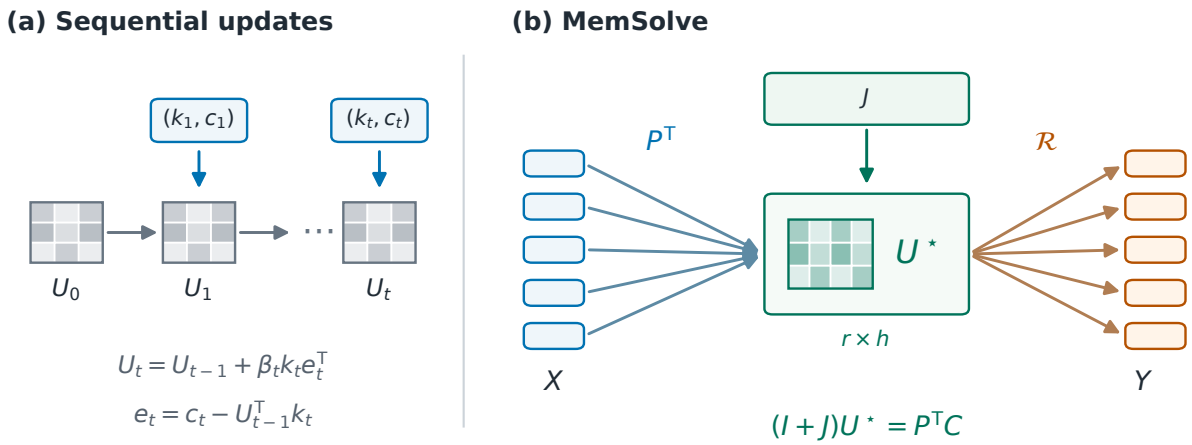


Figure 1: **From sequential memory updates to global equilibrium.** (a) The delta rule corrects one association at a time. (b) MemSolve forms an input-specific equilibrium from all associations. J is learned across samples; P and C depend on the input. The readout is $\mathcal{R}(U^\star) = \eta C + P[2U^\star - (1+\eta)P^\top C]$. This compares memory mechanisms; MemSolve is not an equivalent unrolling of (a). Fig. 2 expands its computation.

1 Introduction

Error-correcting associative memories store contextual associations by predicting their content and updating the memory from the residual. The delta rule implements this mechanism through successive writes, while

test-time training (TTT) treats the state more generally as an internal model adapted to the current context [12, 18, 13].

For visual recognition and other full-context tasks, all associations are available together. An ordered update rule nevertheless makes each correction depend on the memory left by earlier positions, introducing a sequential dependency into a task that permits joint processing. ViT^3 studies this issue in vision and finds that a full-batch inner update can outperform sequential mini-batch updates. More inner steps can improve accuracy, but also reduce throughput and may destabilize training [6]. A natural alternative is to determine a memory that is already consistent with the joint correction field.

Two obstacles must be resolved. First, exposing bidirectional interactions does not by itself make error correction cheap. LION's Appendix B.7 derives bidirectional recurrent and chunkwise DeltaNet forms, but its discussed full-attention expression requires an inverse with cubic cost in sequence length [1]. That expression can therefore lose the computational advantage sought from a compact memory: doubling the sequence length multiplies the inversion cost by eight. This is a limitation of that formulation, rather than an impossibility result for bidirectional DeltaNet. Second, a well-defined memory state is not enough. Its readout must control how a compact correction acts on the much larger token space.

We address these obstacles by designing the memory coordinates, correction system, and readout together. MemSolve maps relation features to a soft frame whose singular values are below one. It then combines a positive-definite component with a skew-symmetric coupling to define an accretive system in rank space. The system has a unique equilibrium; its reflected resolvent supplies the compact readout map. Using the same frame to write and read makes the resulting token operator a scalar content map plus a rank-limited correction, with a spectral norm strictly below one for every realized frame. The solve dimension depends on the chosen rank, not the sequence length.

This design separates *learning the correction law* from *adapting the memory state*. The core is learned across training samples, whereas the frame, contextual statistic, and equilibrium state are computed anew for each input. A sample-conditioned core generator is therefore not required for input-dependent memory. This distinction also has a practical consequence: each head's compact core map can be computed once per forward pass and reused across the batch.

Intention studies global regression with query readout, and MesaNet develops locally optimal test-time regression for causal sequence modeling [5, 15]. Building on this perspective, our contribution is the joint construction of a shared soft frame, an accretive correction law, and a complementary content path. This structure connects three aspects of the mixer: a uniquely solvable memory, a contractive readout with an exact compact spectral characterization, and an implementation whose cost is linear in sequence length at fixed rank. We evaluate it in a non-hierarchical visual backbone: MemSolve-ViT-S reaches 80.95% on ImageNet-1K, while a matched short-run comparison on ImageNet-100 tests the contribution of the learned core.

2 Related Work

Error-correcting and test-time memories. Fast-weight models connect linear sequence layers with associative-memory updates [12]. DeltaNet replaces additive writes with residual correction; its chunkwise formulation makes such updates amenable to parallel training [18]. TTT uses an internal learner as the sequence state [13], and test-time regression provides a common language for several memory update rules [16]. MesaNet solves contextual regression problems with conjugate gradients [15]. MemSolve combines residual correction with a direct full-context solve. Its correction field is defined jointly over the input, rather than by composing causal memory updates.

Global regression and equilibrium layers. Intention maps keys and values to a regularized least-squares solution and reads that solution with queries [5]. MemSolve couples the write and read coordinates through a normalized frame and extends the compact equilibrium map with a complementary content path. Deep equilibrium models and monotone equilibrium networks also obtain representations through fixed points [2, 17]. Here the equilibrium is a compact linear system with a direct factorization. The Cayley transform and its relation to accretive operators are classical [3]; we use this relation to control the realized token mixer.

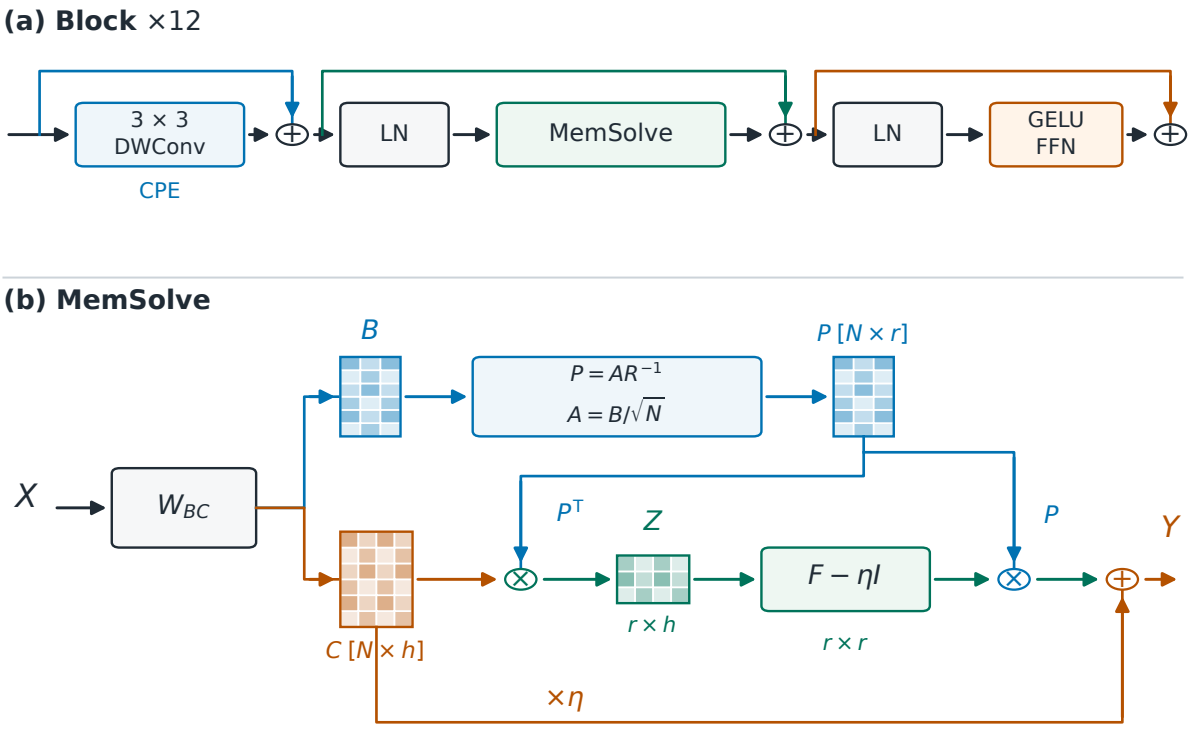


Figure 2: **MemSolve-ViT and its token mixer.** (a) The visual block has residual CPE, a pre-LayerNorm MemSolve branch, and a pre-LayerNorm GELU FFN branch; stochastic depth on the latter two branches is omitted from the drawing. (b) One head projects relations B and content C, forms the soft frame P, writes $Z = P^{\mathsf{T}}C$, applies the shared core correction, and reads through P before adding ηC. Circled multiplication symbols denote matrix products. Head concatenation and the output projection follow this computation; F is defined in Eq. (5). The figure shows the mathematical factorization; CUDA evaluates compact statistics without storing the token-sized frame.

Bidirectional efficient mixers. Linear attention reorganizes feature interactions to avoid a quadratic attention matrix [10]. Vision Mamba adapts selective state-space modeling to visual inputs, Hydra constructs bidirectional matrix mixers, and LION relates full-attention, recurrent, and chunkwise bidirectional forms [19, 8, 1]. MemSolve instead obtains a joint memory through sufficient statistics and a rank-space solve. Its operator does not prescribe a spatial scan order. For vision, we supply local spatial structure through conditional positional encoding (CPE) [4] in a plain backbone derived from ViT^3 [6].

3 MemSolve

3.1 Relation features and a soft frame

Consider one head with N valid tokens, relation rank r, and content width h. A learned projection produces relation features $B \in \mathbb{R}^{N\times r}$ and content $C \in \mathbb{R}^{N\times h}$ from the input X. Define

$$A = \frac{B}{\sqrt{N}}, \qquad R^{\mathsf{T}}R = \mathbf{I}_r + A^{\mathsf{T}}A, \qquad P = AR^{-1}, \qquad Z = P^{\mathsf{T}}C, \tag{1}$$

where R is the upper Cholesky factor with positive diagonal. The same P writes content into the statistic $Z \in \mathbb{R}^{r\times h}$ and reads a compact state back to tokens. There is no independent query projection. The $N^{-1/2}$ factor makes $A^{\mathsf{T}}A$ an average relation second moment instead of an unnormalized sum.

The identity term in Eq. (1) keeps the factorization positive definite even when the relations are rank deficient. It also gives

$$\sigma_i(P) = \frac{\sigma_i(A)}{\sqrt{1+\sigma_i(A)^2}} < 1. \tag{2}$$

We call P a *soft frame*: it is a contractive relation map, but its columns are not generally orthonormal. In particular, PP^T and $\mathbf{I}_N - PP^\mathsf{T}$ are not orthogonal projectors.

3.2 A learned correction law and an input-dependent state

Each head learns a matrix $\Theta \in \mathbb{R}^{r\times r}$ shared across samples. Its lower and upper triangular entries parameterize

$$L = \mathrm{tril}(\Theta, -1) + \mathrm{diag}(\mathrm{softplus}(\mathrm{diag}\,\Theta + \log(e-1))), \tag{3}$$

$$\Omega = \mathrm{triu}(\Theta, 1) - \mathrm{triu}(\Theta, 1)^\mathsf{T}, \qquad J = LL^\mathsf{T} + \Omega. \tag{4}$$

Since L has a positive diagonal, $LL^\mathsf{T} \succ 0$; since $\Omega^\mathsf{T} = -\Omega$, it contributes coupling without changing the symmetric part of J. The equilibrium memory and compact map are

$$U^\star = (\mathbf{I}_r + J)^{-1}Z, \qquad F = (\mathbf{I}_r + J)^{-1}(\mathbf{I}_r - J) = 2(\mathbf{I}_r + J)^{-1} - \mathbf{I}_r. \tag{5}$$

In implementations, inverse notation denotes a linear solve. J and F are shared across samples; P, Z, and $U^\star$ are not.

To see the error-correction interpretation, define

$$\Gamma_X = \mathbf{I}_r + LL^\mathsf{T} - P^\mathsf{T}P \succ 0. \tag{6}$$

The memory dynamics become

$$\dot{U} = \underbrace{P^\mathsf{T}(C - PU)}_{\text{association residuals}} - \underbrace{\Gamma_X U}_{\text{regularization}} - \underbrace{\Omega U}_{\text{skew coupling}} \tag{7}$$

$$= Z - (\mathbf{I}_r + J)U. \tag{8}$$

The residual term sums all positions' prediction errors against the same state. When $\Omega = 0$, its equilibrium minimizes

$$\mathcal{E}_X(U) = \tfrac{1}{2}\,\|PU - C\|_F^2 + \tfrac{1}{2}\,\mathrm{tr}(U^\mathsf{T}\Gamma_X U). \tag{9}$$

For nonzero Ω, the equilibrium solves the strongly monotone linear equation $(\mathbf{I}_r + J)U = Z$, while the dynamics need not be a gradient flow of this scalar objective.

The dependence of Γ_X on P is deliberate. It cancels the frame Gram matrix in the total correction system, leaving the learned matrix $\mathbf{I}_r + J$. This is a structured regularized-memory construction, not ordinary ridge regression with an independently chosen scalar penalty. It is also why the solve can be shared across samples without making their memories identical. The dynamics explain the state; the forward pass computes that state directly and does not choose an inner step count or learning rate.

3.3 Content plus a compact correction

The solve in Eq. (5) maps the contextual statistic Z to $U^\star$. Reflecting this resolvent gives $2U^\star - Z = FZ$, a compact contraction (Theorem 4.2). We lift this map through the same frame used to write the memory and retain a content response through $\mathbf{I}_N - PP^\mathsf{T}$. With a learned per-head scalar $\eta = \tanh(\eta_{\mathrm{raw}}) \in (-1, 1)$, the resulting readout is

$$Y = \eta C + P\,[2U^\star - (1+\eta)Z] \tag{10}$$

$$= \underbrace{\eta C}_{\text{content term}} + \underbrace{P(F - \eta\mathbf{I}_r)P^\mathsf{T}C}_{\text{rank-limited correction}}. \tag{11}$$

Equivalently, $Y = \eta(\mathbf{I}_N - PP^\mathsf{T})C + PFP^\mathsf{T}C$. The factor two is inherited from the reflected resolvent in Eq. (5); it is not an arbitrary output scale. Head outputs are concatenated and passed through a learned output projection.

The default EQUILIBRIUM mode learns F through J. The COMPLEMENT mode fixes $F = 0$ and omits the core parameters:

$$Y_{\text{com}} = \eta(\mathbf{I}_N - PP^\mathsf{T})C = \eta(\mathbf{I}_N + AA^\mathsf{T})^{-1}C. \tag{12}$$

It is a nonzero, input-dependent mixer. The identity on the right is useful for analysis; long sequences do not require a token-space inverse. At initialization, $\Theta = 0$ gives $L = \mathbf{I}_r$, $J = \mathbf{I}_r$, and $F = 0$. With identical remaining parameters, both modes therefore start from the same function. The memory itself is $U^\star = Z/2$, not zero. Equilibrium can subsequently learn a compact correction, while Complement continues to learn the frame, content, output projection, and η. We initialize η to 0.9.

4 Properties of the Operator

All statements in this section concern one head in exact arithmetic. Proofs are in Appendix A.

Proposition 4.1 (Well-posed memory)**.** *Let $\mu = 1 + \sigma_{\min}(L)^2 > 1$. Then Eq. (8) has the unique equilibrium in Eq. (5), and*

$$\left\|(\mathbf{I}_r + J)^{-1}\right\|_2 \le \mu^{-1}, \qquad \left\|(\mathbf{I}_r + J)^{-\mathsf{T}}\right\|_2 \le \mu^{-1}. \tag{13}$$

Every trajectory obeys $\|U(t) - U^\star\|_F \le e^{-\mu t}\|U(0) - U^\star\|_F$.

The bound controls sensitivity to the right-hand side in both the forward and adjoint solves. The full parameter gradients additionally depend on the projections, frame, and core parameterization; the system's condition number also depends on $\|\mathbf{I}_r + J\|_2$.

Theorem 4.2 (Contractive compact and token maps)**.** *The map F in Eq. (5) satisfies*

$$\mathbf{I}_r - F^\mathsf{T}F = 4(\mathbf{I}_r + J)^{-\mathsf{T}}LL^\mathsf{T}(\mathbf{I}_r + J)^{-1} \succ 0. \tag{14}$$

For a realized frame P, define

$$K_X = \eta(\mathbf{I}_N - PP^\mathsf{T}) + PFP^\mathsf{T}. \tag{15}$$

Then $\|K_X\|_2 \le \max\{|\eta|, \|F\|_2\} < 1$ and $\operatorname{rank}(K_X - \eta\mathbf{I}_N) \le r$. Moreover, $K_X v = \eta v$ for every $v \in \ker(P^\mathsf{T})$.

This makes the division of work explicit: the content path preserves a scaled response outside the relation span, while the learned core changes interactions within that span. Sharing the write and read frame does not force symmetry: when F is nonsymmetric, PFP^T may be nonsymmetric too. The accretive Cayley parameterization covers every real strict contraction in r dimensions (Appendix A.3); the rank and shared-frame structure still constrain the token operator.

The norm bound is conditional on the realized frame. The nonlinear map $X \mapsto K_X C(X)$ also differentiates through $P(X)$, and the surrounding output projection and residual connection are unrestricted. We therefore do not interpret Theorem 4.2 as a global Lipschitz certificate for the model or as a proof of generalization.

Proposition 4.3 (Exact analysis in compact space)**.** *Let $P = QSV^\mathsf{T}$ be a thin singular value decomposition with $k = \operatorname{rank}(P)$, $Q^\mathsf{T}Q = V^\mathsf{T}V = \mathbf{I}_k$, and positive diagonal S. The token map is orthogonally similar to*

$$\left[\eta\mathbf{I}_k + SV^\mathsf{T}(F - \eta\mathbf{I}_r)VS\right] \oplus \eta\mathbf{I}_{N-k}. \tag{16}$$

Its singular values, including its exact spectral norm, can therefore be obtained from a matrix of size at most r, together with $N - k$ copies of $|\eta|$.

Without a positional module, permuting input rows permutes the output rows: $A^\mathsf{T}A$ and the shared core are unchanged. This is useful for unordered full-context aggregation, but it does not encode positions by itself. The visual backbone introduces spatial information separately through CPE.

Table 1: **MemSolve-ViT configurations.** Parameters include the 1,000-class head and the Equilibrium core. All models have depth 12 and patch size 16; DropPath increases linearly to the listed maximum.

Model	Width	Heads	Rank/head	FFN width	DropPath	Params (M)
Tiny	192	6	16	768	0.0	5.054
Small	384	6	32	1584	0.1	19.876
Base	768	12	48	3072	0.4	77.981

5 Compact Computation

The forward pass can eliminate the frame in Eq. (1) by forming the sufficient statistics

$$G = A^{\mathsf{T}}A, \qquad H = A^{\mathsf{T}}C, \qquad R^{\mathsf{T}}R = \mathbf{I}_r + G. \tag{17}$$

The forward pass is exactly

$$Z = R^{-\mathsf{T}}H, \qquad W = R^{-1}(F - \eta\mathbf{I}_r)Z, \qquad Y = \eta C + AW. \tag{18}$$

This eliminates the token-sized intermediate P and computes F once per head per forward pass. The core is recomputed on each forward pass. Complement can instead solve $(\mathbf{I}_r + G)D = H$ and use $Y = \eta(C - AD)$, avoiding the core solve entirely. The current CUDA implementation uses a common factor-based computation for both modes: it constructs $T = R^{-\mathsf{T}}$ once by solving triangular systems with identity right-hand sides, then evaluates $Z = TH$ and $W = T^{\mathsf{T}}(F - \eta\mathbf{I}_r)Z$, with $F = 0$ for Complement. This replaces repeated triangular dependencies with compact matrix products without changing the operator.

Per head, the mixer costs $O(Nr^2 + Nrh + r^3 + r^2h)$, with token storage linear in N and compact storage $O(r^2 + rh)$. Input and output projections add their usual dense-linear costs, also linear in N for fixed model dimensions. These bounds concern sequence scaling; increasing rank still raises the cost of statistics and factorizations. When physical $N \le r$, the implementation evaluates the complementary term using the smaller positive-definite system $\eta(\mathbf{I}_N + AA^{\mathsf{T}})^{-1}C$. This algebraically equivalent branch avoids subtraction of nearly equal terms for short, strong-relation inputs and never creates a system larger than the supported rank.

Our CUDA implementation combines tiled token statistics and readout kernels with library factorizations and compact solves. It uses Triton for token operations and selective fusion, cuSOLVERDx for small factorizations and solves, and cuBLASLt for projection gradients. The backward pass uses analytic first-order derivatives for the rank-space path and reuses compact factors. The short-sequence path differentiates its direct solves; neither path backpropagates through an unrolled memory trajectory. BF16 token operands are accumulated in FP32. Compact matrices and solves remain in FP32, while sensitive readout coefficients are represented by a BF16 leading part and a BF16 residual part, following the multiword arithmetic principle of Henry et al. [7]. The output follows the activation dtype. Algorithms 1–3 in Appendix B specify the forward computation, rank-space backward pass, and shared-core gradients.

6 Experiments

We evaluate MemSolve-ViT-S on ImageNet-1K and study its learned core on a fixed ImageNet-100 subset. Table 1 also lists Tiny and Base configurations; all classification results below use the Small backbone.

6.1 Visual backbone and training

MemSolve-ViT uses non-overlapping 16×16 patches and twelve blocks at a constant width. Each block first adds a residual 3×3 depthwise CPE, then applies a pre-LayerNorm MemSolve branch and a pre-LayerNorm GELU FFN branch, each with a residual connection and stochastic depth. A final per-token LayerNorm precedes mean pooling and a linear classifier. There is no class token, absolute position table, LayerScale, or internal rotary transform. Table 1 specifies the configurations.

We train from scratch for 300 epochs at 224×224 on ImageNet-1K [11]. The recipe is derived from the plain ViT[3] training setup, with the batching and augmentation changes detailed below. Both completed

Table 2: **Published non-hierarchical Small backbones and MemSolve.** All entries use 224×224 inputs. Baseline numbers are taken from the cited papers; these are not matched-training-protocol comparisons. $\natural$ denotes LION's multiple scans. Vim-S† additionally uses 30 epochs of long-sequence fine-tuning. MemSolve values use the best validation checkpoint of each completed 300-epoch run.

Model	Mixer family	Params (M)	Top-1 (%)
DeiT-S [14]	Self-attention	22	79.8
LION-d$^\natural$-S [1]	Bidirectional linear recurrence	22	79.9
Vim-S [19]	Bidirectional state space	26	80.3
Vim-S† [19]	Bidirectional state space	26	81.4
ViT^3-S [6]	Test-time training	24	81.6
MemSolve-ViT-S	Associative-memory equilibrium	19.88	80.79
MemSolve-ViT-S (RA, batch 2,048)	Associative-memory equilibrium	19.88	80.95

Table 3: **Completed Small runs on ImageNet-1K.** CE is hard-label validation cross-entropy. "Best" refers to Top-1 selection; the CE shown in that column comes from the same checkpoint, not a separately selected minimum. Each row is one seed-0 run.

Configuration	Best epoch	Best Top-1	CE at best	Final Top-1	Final CE
Batch 1,024, no RA	294	80.785	0.8556	80.680	0.8576
Batch 2,048, RA	300	80.948	0.8419	80.948	0.8419

Small runs use fused AdamW, weight decay 0.05, gradient clipping at norm 5, cross-entropy with label smoothing 0.1, Mixup 0.8, CutMix 1.0, RandAugment, and random erasing 0.25. The learning rate warms up linearly for 20 epochs and then decays with an update-based cosine schedule. Validation resizes the shorter side to 256 and takes a 224×224 center crop. Both runs use seed 0 and evaluate ordinary model weights without EMA. We report the best validation Top-1 checkpoint, and provide final-epoch results separately.

The baseline uses global batch 1,024 and peak learning rate 10^{-3}. The second run uses three-view repeated augmentation (RA), global batch 2,048, and peak learning rate 2×10^{-3}. Each GPU processes 512 images per microbatch; the second run accumulates two microbatches on each of two A800 GPUs. Mixup/CutMix acts on the complete physical microbatch. Repeated augmentation keeps an approximately 1.28M-presentation epoch budget instead of tripling it. The second run also changes sample ordering. Consequently, this comparison measures the combined training configuration, not the isolated causal effect of RA.

6.2 ImageNet-1K classification

Table 2 places MemSolve among published plain backbones. The batch-1,024 configuration reaches 80.79% Top-1, while the RA/batch-2,048 configuration reaches 80.95% with 19.88M parameters. The latter is above the paper-reported Vim-S result and below Vim-S† and ViT^3-S. This is a comparison of complete models under their respective recipes; it does not isolate the effect of replacing the token mixer.

The combined RA/batching change improves best accuracy by 0.163 percentage points and lowers validation CE at the selected checkpoint (Table 3). Figure 3 includes the full validation trajectories. These observations describe two training runs; they do not estimate seed-to-seed uncertainty.

6.3 Does the learned core contribute?

We compare Equilibrium with Complement on the 100 ImageNet classes used in the public CMC class list[1]. Both use the Small backbone, a 100-class classifier, rank 32, seed 0, and the same initialization for every shared parameter. The two modes have the same initial mathematical function, since $F = 0$; only Equilibrium has trainable core parameters. We train each for 30 epochs with batch 512, a five-epoch warmup, peak

[1] `https://github.com/HobbitLong/CMC/blob/master/imagenet100.txt`

Table 4: **Learning the compact core on ImageNet-100.** Both modes reach their best Top-1 at epoch 30. This is a single-seed, 30-epoch comparison with matched shared initialization, not a converged ImageNet-1K comparison.

Mode	Params (M)	Top-1 (%)	Top-5 (%)	Validation CE
Complement ($F = 0$)	19.456	62.84	87.74	1.4929
Equilibrium (learned F)	19.530	64.54	88.76	1.4541

learning rate 5×10^{-4}, and a full 30-epoch cosine schedule. Other augmentation and optimizer settings follow the no-RA configuration above.

Equilibrium improves Top-1 by 1.70 percentage points and reduces validation CE (Table 4). Its additional 73,728 parameters are the 12 layers' six rank-32 cores. Starting from the same mathematical function, learning the compact map therefore improves the 30-epoch outcome in this setting. Longer runs and component ablations would be needed to determine the gain at convergence and the separate contributions of symmetric and skew coupling.

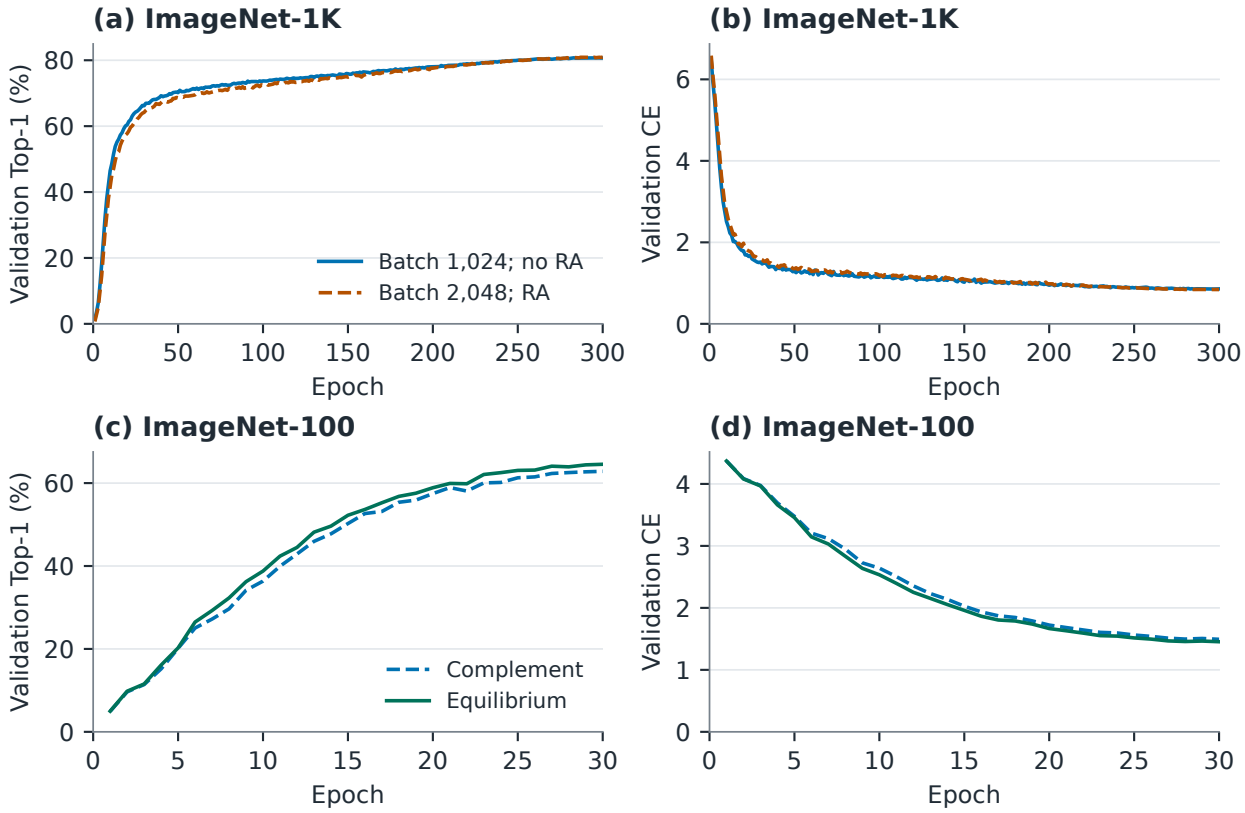


Figure 3: **Validation trajectories from completed runs.** Top: the two 300-epoch ImageNet-1K Small configurations. Bottom: the 30-epoch ImageNet-100 core comparison. Every recorded epoch is plotted without smoothing, using ordinary model weights. Curves show one seed per configuration, not means over repeated runs. The RA comparison also changes effective batch, learning-rate scale, and sample ordering.

7 Discussion and Limitations

MemSolve concentrates global interaction in a learned relation span. This gives a direct computational tradeoff: larger ranks can represent more independent interaction directions, but increase statistics and

factorization costs. The shared frame also couples writing and reading, whereas separate query and key projections permit different association geometries. The present results support the shared construction for image classification; tasks requiring distinct write and read coordinates remain an open direction.

The shared solve relies on co-designing the regularizer with the frame. Changing the memory objective may require a different factorization and readout analysis. Empirically, the present evaluation covers a Small visual backbone and a short core comparison. Larger models, language tasks, and longer sequences will test how broadly this construction applies. Repeated runs are also needed to quantify the training variance of the reported single-seed results.

8 Conclusion

MemSolve constructs a full-context associative memory by solving a compact correction system. A shared soft frame connects memory formation to a content-plus-correction readout, allowing the solve, contraction, and token spectrum to be analyzed within one formulation. The learned core is shared across samples while the memory remains input-dependent. Compact statistics make this formulation executable without a token-sized frame, and the completed image-classification experiments demonstrate that the resulting plain backbone is competitive at Small scale.

A Proofs and Structural Details

A.1 Soft-frame geometry

Since $R^\mathsf{T}R = \mathbf{I} + A^\mathsf{T}A$,

$$PP^\mathsf{T} = A(\mathbf{I} + A^\mathsf{T}A)^{-1}A^\mathsf{T}. \tag{19}$$

An SVD of A gives Eq. (2). In particular, $P^\mathsf{T}P \prec \mathbf{I}_r$ and $PP^\mathsf{T} \prec \mathbf{I}_N$, including rank-deficient cases. The Woodbury identity yields

$$\mathbf{I}_N - PP^\mathsf{T} = (\mathbf{I}_N + AA^\mathsf{T})^{-1} \succ 0, \tag{20}$$

which proves Eq. (12) and positivity of Γ_X. The reduced QR factorization of $[A^\mathsf{T}, \mathbf{I}_r]^\mathsf{T}$ with a positive diagonal in its triangular factor has top block P. Thus the QR reference and the compact Cholesky construction agree in exact arithmetic.

Normalization by $\sqrt{N}$ has a specific, limited role. If $N^{-1}\sum_i \|b_i\|_2^2 \le c$, then $\|A\|_2^2 \le \|A\|_F^2 \le c$ and $\kappa_2(\mathbf{I} + A^\mathsf{T}A) \le 1 + c$, independently of N. This conditional bound prevents sequence length alone from scaling the Gram matrix; it does not assume that feature energy is bounded throughout training or imply accuracy under length extrapolation.

A.2 Memory existence, sensitivity, and convergence

Write $M = \mathbf{I}_r + J$. For every real v,

$$v^\mathsf{T}Mv = v^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})v \ge \mu\,\|v\|_2^2, \qquad \mu = 1 + \sigma_{\min}(L)^2. \tag{21}$$

Cauchy–Schwarz gives $\|Mv\|_2 \ge \mu\,\|v\|_2$, so M is invertible and $\left\|M^{-1}\right\|_2 \le 1/\mu$. The same argument applies to M^T. For $E(t) = U(t) - U^\star$, $\dot{E} = -ME$ and

$$\frac{d}{dt}\,\|E\|_F^2 = -2\,\mathrm{tr}(E^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})E) \le -2\mu\,\|E\|_F^2\,. \tag{22}$$

Integration proves Proposition 4.1. For $\Omega = 0$, differentiating Eq. (9) gives $\nabla_U \mathcal{E}_X = MU - Z$ and Hessian $M \succ 0$ for each content column. When $\Omega \neq 0$, the nonsymmetric field need not be a scalar gradient, although the same convergence argument holds.

A.3 Cayley contraction and coverage

As $M = \mathbf{I} + J$ commutes with $\mathbf{I} - J$,

$$\mathbf{I} - F^\mathsf{T}F = M^{-\mathsf{T}}\left[M^\mathsf{T}M - (\mathbf{I} - J)^\mathsf{T}(\mathbf{I} - J)\right]M^{-1} \tag{23}$$

$$= 2M^{-\mathsf{T}}(J + J^\mathsf{T})M^{-1} = 4M^{-\mathsf{T}}LL^\mathsf{T}M^{-1} \succ 0. \tag{24}$$

Conversely, let F_0 be any real matrix with $\|F_0\|_2 < 1$. Then $\mathbf{I} + F_0$ is invertible and

$$J_0 = (\mathbf{I} - F_0)(\mathbf{I} + F_0)^{-1}, \qquad \mathrm{sym}(J_0) = (\mathbf{I} + F_0)^{-\mathsf{T}}(\mathbf{I} - F_0^\mathsf{T}F_0)(\mathbf{I} + F_0)^{-1} \succ 0. \tag{25}$$

Factor $\mathrm{sym}(J_0) = LL^\mathsf{T}$ and set $\Omega = (J_0 - J_0^\mathsf{T})/2$. The triangular/softplus parameterization can represent every such L and Ω. This establishes coverage of the strict contraction ball in compact space. It is an application of the classical accretive–Cayley correspondence, not an unrestricted token-map universality statement.

A.4 Token contraction and the exact compact spectrum

Let $D = (\mathbf{I}_N - PP^\mathsf{T})^{1/2}$ and $V_0 = [D^\mathsf{T}, P]^\mathsf{T} \in \mathbb{R}^{(N+r)\times N}$, so $V_0^\mathsf{T}V_0 = \mathbf{I}_N$. Then

$$K_X = V_0^\mathsf{T}\,\mathrm{diag}(\eta\mathbf{I}_N, F)V_0, \tag{26}$$

which proves the norm bound in Theorem 4.2. The rank and nullspace statements follow directly from $K_X - \eta\mathbf{I}_N = P(F - \eta\mathbf{I}_r)P^\mathsf{T}$.

For Proposition 4.3, extend Q to an orthogonal basis $[Q, Q_\perp]$. In this basis the low-rank correction acts only on the first k coordinates, giving Eq. (16). If $k = 0$, the result is simply $\eta \mathbf{I}_N$; if $k = N$, the complementary block is absent. One may compute V and S from the positive eigenspace of $P^\mathsf{T} P$, so no $N \times N$ matrix or token-space SVD is required.

B Algorithms and Numerical Details

B.1 Forward computation

Algorithm 1 gives the computation for one head across a batch; heads can be evaluated independently. The projections into B, C and the output projection are ordinary affine layers. The algorithm returns head outputs, which are concatenated, projected, and masked again so that an output bias cannot populate padded positions.

We distinguish physical length N from the number n_b of valid tokens in sample b. Masks multiply rows and are held fixed under differentiation. We use $\mathrm{solve}(M, V)$ for the solution of $MU = V$, $\mathrm{chol}_u(S)$ for the upper factor R such that $R^\mathsf{T} R = S$, and

$$\mathrm{cholsolve}(R, V) = \mathrm{solve}\big(R, \mathrm{solve}(R^\mathsf{T}, V)\big)\,. \tag{27}$$

The solve notation specifies exact-arithmetic dependencies; a general core system uses an LU factorization. CUDA constructs the inverse triangular factor $T = R^{-\mathsf{T}}$ with library triangular solves and reuses it in forward and backward products. It does not form a dense inverse of the token-space operator. The pseudocode also gives the equivalent Complement-specific derivative for clarity; the implementation shares the inverse-factor adjoint between modes. For lower factor $L_R = R^\mathsf{T}$, set $M_F = F - \eta \mathbf{I}_r$, $U = TE_W$, and $V = UZ^\mathsf{T}$. The common compact adjoints are $\overline{H} = T^\mathsf{T} M_F^\mathsf{T} U$ and $\overline{G} = T^\mathsf{T}\,\mathrm{sym}_\ell(-M_F V^\mathsf{T} - M_F^\mathsf{T} V)T$, where $\mathrm{sym}_\ell(K) = [\mathrm{tril}(K) + \mathrm{tril}(K, -1)^\mathsf{T}]/2$. The shared-core and scalar adjoints reuse V. Appendix B.3 gives the execution precision and fusion choices.

Algorithm 1 MemSolve forward without materializing P (one head)

Input: $\{B_b \in \mathbb{R}^{N\times r}, C_b \in \mathbb{R}^{N\times h}, m_b \in \{0,1\}^N\}_{b=1}^s$; mode, η; $\Theta \in \mathbb{R}^{r\times r}$ for Equilibrium
Output: Head outputs $\{Y_b \in \mathbb{R}^{N\times h}\}_{b=1}^s$

if mode is Equilibrium **then**
  Construct L, Ω, J from Θ using Eq. (4)
  $M \leftarrow \mathbf{I}_r + J$; factor M once
  $F \leftarrow \mathrm{solve}(M, \mathbf{I}_r - J)$ ▷ Shared across this batch
end if
for each sample b in parallel **do**
  $n_b \leftarrow \sum_i m_{b,i}$; $A \leftarrow (m_b \odot B_b)/\sqrt{\max(n_b, 1)}$; $C \leftarrow m_b \odot C_b$
  if $N \le r$ **then** ▷ Physical length determines the branch
    $R_N \leftarrow \mathrm{chol}_u(\mathbf{I}_N + AA^\mathsf{T})$; $Y_b \leftarrow \eta\,\mathrm{cholsolve}(R_N, C)$
    if mode is Equilibrium **then**
      $R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + A^\mathsf{T} A)$; $Z \leftarrow \mathrm{solve}(R^\mathsf{T}, A^\mathsf{T} C)$
      $Y_b \leftarrow Y_b + A\,\mathrm{solve}(R, FZ)$
    end if
  else
    $G \leftarrow A^\mathsf{T} A$; $H \leftarrow A^\mathsf{T} C$; $R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$
    if mode is Equilibrium **then**
      $Z \leftarrow \mathrm{solve}(R^\mathsf{T}, H)$; $W \leftarrow \mathrm{solve}(R, (F - \eta \mathbf{I}_r)Z)$
    else ▷ Complement
      $D \leftarrow \mathrm{cholsolve}(R, H)$; $W \leftarrow -\eta D$
    end if
    $Y_b \leftarrow \eta C + AW$
  end if
end for
return $\{Y_b\}_{b=1}^s$

For $N > r$, backward needs A, C, R, W and either Z or D; these can be saved or reconstructed from saved projections and compact quantities. Equilibrium also reuses F, L, and the factorization of M across the batch. No token-sized frame is needed. For $N \le r$, the implementation differentiates the short direct-solve expression using automatic differentiation, with an analytic adjoint for the shared linear solve. The largest factorization in either branch is at most $r \times r$.

Initialization and scalar parameterization. An all-masked input uses a safe divisor of one and yields zero. The floating-point implementation uses an interior form of tanh for η to avoid rounding to the endpoints. The theoretical parameterization uses the open interval $(-1, 1)$. Core initialization $\Theta = 0$ does not remove its training signal: $\delta F = -\frac{1}{2}\delta J$ at $J = \mathbf{I}_r$, so the core can receive gradients although its initial forward correction $PFP^{\mathsf{T}}C$ is zero.

B.2 Rank-space backward and shared-core gradients

Algorithm 2 differentiates the $N > r$ branch for one sample and head. A bar denotes the adjoint of a variable under the Frobenius inner product, and $\mathrm{sym}(T) = (T + T^{\mathsf{T}})/2$. The algorithm includes both the direct content term and the correction, so that their contributions to η and C are accumulated together. Projection gradients then follow the usual affine-layer rules.

For an upper Cholesky factor R, define

$$\Phi_u(T) = \mathrm{triu}(T, 1) + \tfrac{1}{2}\,\mathrm{diag}(\mathrm{diag}\, T), \qquad \mathcal{C}(R, E_R) = R^{-1}\,\mathrm{sym}\big(\Phi_u(E_R R^{\mathsf{T}})\big)\,R^{-\mathsf{T}}. \tag{28}$$

Here $\mathcal{C}$ is the adjoint of $R = \mathrm{chol}_u(S)$ with respect to the symmetric matrix S; the inverse notation again means two triangular solves. To obtain it, differentiate $S = R^{\mathsf{T}}R$ and observe that $dR = \Phi_u(R^{-\mathsf{T}}(dS)R^{-1})R$. This also fixes the half-diagonal convention in Eq. (28).

Algorithm 2 Rank-space backward ($N > r$, one sample and head)

Input: $E = \overline{Y} \in \mathbb{R}^{N\times h}$; forward values A, C, R, W, η; mode; F, Z for Equilibrium or D for Complement; fixed mask m and count n
Output: $\overline{B}, \overline{C}_{\mathrm{in}}$; per-sample contributions $\overline{\eta}, \overline{F}$

$E_W \leftarrow A^{\mathsf{T}}E$; $\overline{A} \leftarrow EW^{\mathsf{T}}$; $\overline{C} \leftarrow \eta E$
$\overline{\eta} \leftarrow \langle E, C\rangle_F$; $\overline{F} \leftarrow 0$
if mode is Equilibrium **then**
  $E_T \leftarrow \mathrm{solve}(R^{\mathsf{T}}, E_W)$
  $\overline{F} \leftarrow E_T Z^{\mathsf{T}}$; $\overline{\eta} \leftarrow \overline{\eta} - \langle E_T, Z\rangle_F$
  $\overline{Z} \leftarrow (F - \eta\mathbf{I}_r)^{\mathsf{T}}E_T$; $\overline{H} \leftarrow \mathrm{solve}(R, \overline{Z})$
  $\overline{R} \leftarrow -E_T W^{\mathsf{T}} - Z\overline{H}^{\mathsf{T}}$
  $\overline{G} \leftarrow \mathcal{C}(R, \overline{R})$ ▷ Eq. (28)
else ▷ Complement: $D = (\mathbf{I}_r + G)^{-1}H$
  $T \leftarrow \mathrm{cholsolve}(R, E_W)$
  $\overline{H} \leftarrow -\eta T$; $\overline{G} \leftarrow \eta\,\mathrm{sym}(TD^{\mathsf{T}})$
  $\overline{\eta} \leftarrow \overline{\eta} - \langle E_W, D\rangle_F$
end if
$\overline{A} \leftarrow \overline{A} + C\overline{H}^{\mathsf{T}} + A(\overline{G} + \overline{G}^{\mathsf{T}})$
$\overline{C} \leftarrow \overline{C} + A\overline{H}$
$\overline{B} \leftarrow m \odot \overline{A}/\sqrt{\max(n, 1)}$; $\overline{C}_{\mathrm{in}} \leftarrow m \odot \overline{C}$
return $\overline{B}, \overline{C}_{\mathrm{in}}, \overline{\eta}, \overline{F}$

For $F = M^{-1}(\mathbf{I} - J)$, differentiation gives

$$dF = -M^{-1}(dJ)(\mathbf{I} + F). \tag{29}$$

Since the core is shared, its adjoints are summed over samples before the transposed solve in Algorithm 3. The reduction is a sum: any loss averaging is already contained in $\overline{Y}$. The fast path accumulates this reduction in tiles rather than storing one $r \times r$ core adjoint per sample.

Algorithm 3 Shared-core backward (once per head, Equilibrium only)

Input: $E_F = \sum_b \overline{F}_b$; Θ, L, F and the saved factorization of $M = \mathbf{I}_r + J$
Output: Core-parameter gradient $\overline{\Theta} \in \mathbb{R}^{r \times r}$

$T \leftarrow \mathrm{solve}(M^{\mathsf{T}}, E_F)$ ▷ Reuse the forward LU factors
$\overline{J} \leftarrow -T(\mathbf{I}_r + F)^{\mathsf{T}}$
$\overline{L} \leftarrow (\overline{J} + \overline{J}^{\mathsf{T}})L$
$d \leftarrow \mathrm{diag}(\overline{L}) \odot \mathrm{sigmoid}(\mathrm{diag}\,\Theta + \log(e-1))$
$\overline{\Theta} \leftarrow \mathrm{tril}(\overline{L}, -1) + \mathrm{diag}(d) + \mathrm{triu}(\overline{J} - \overline{J}^{\mathsf{T}}, 1)$
return $\overline{\Theta}$

For either mode, $\overline{\eta_{\mathrm{raw}}} = (d\eta/d\eta_{\mathrm{raw}}) \sum_b \overline{\eta}_b$. The mathematical derivative is $1 - \tanh^2(\eta_{\mathrm{raw}})$. The implementation evaluates the interior-scaled value with native tanh and its derivative through the exponential identity. This retains small values near zero and representable tail derivatives after tanh itself rounds to an endpoint. The implementation supports first-order training gradients; higher-order differentiation is not provided by the fast path.

B.3 Precision and scheduling

The fast path supports ranks 16, 32, 48, and 64. Token projections and statistics use BF16 multiplicands and FP32 accumulation. Gram matrices, compact states, factorization results, and adjoints retain FP32 storage. Batched compact products use Triton's three-term TF32 decomposition; small shared-core products and solves retain IEEE FP32 arithmetic. Readout kernels split sensitive FP32 coefficients into BF16 leading and residual words before accumulating both products in FP32. This retains additional coefficient precision without promoting token-sized matrix products to FP32 operands. Completely masked inputs are covered explicitly. Very sparse masks with strong relations can still amplify relative cancellation error in the rank-space complementary readout; this is a finite-precision boundary, not a change in the exact operator.

Measured implementation scope. The completed training runs used NVIDIA A800-SXM4-80GB GPUs, PyTorch 2.14.0 with CUDA 13.2, and timm 1.0.29. Compilation fuses surrounding operations; CUDA Graph replay reduces launch overhead. These execution choices do not alter Eqs. (1)–(11). This draft reports classification measurements rather than transferring latency numbers from older operator implementations or claiming a matched throughput advantage over other mixers.

C Experimental Details and Provenance

Optimization and augmentation. Small uses FFN ratio $1584/384 = 4.125$; Tiny and Base use ratio 4. The ratio 4 appearing in the shared recipe defaults does not override the Small-specific width. LayerNorm uses $\epsilon = 10^{-6}$. RandAugment is `rand-m9-mstd0.5-inc1`, color jitter is 0.4, and Mixup/CutMix uses application probability 1 and switching probability 0.5. The batch-1,024 run warms up from 10^{-6} and decays to 10^{-5}; the batch-2,048 run uses 2×10^{-6} and 2×10^{-5}. There are 1,251 and 625 optimizer updates per epoch, respectively. The latter uses timm's three-repeat augmentation convention with a fixed epoch budget, global sample permutation, and rank-strided assignment. Persistent data-loader workers receive epoch-dependent random states.

Core comparison. The ImageNet-100 class-list SHA-256 is

`79f3f1552ccc478ce24bd934b4573625db5`
`84ddc1f4643ae23c994a94cb11397`

The run uses 247 optimizer updates, or 126,464 training presentations, per epoch. The minimum learning rate is 10^{-5} and warmup starts at 10^{-6}. Shared weights were checked for equality before training. The initial FP64 reference logits agree exactly in the recorded check; the two BF16 execution paths have a relative logit difference of about 0.235% because their algebraically equivalent computations round differently. This

distinction does not imply different initialized mathematical operators, but it precludes a bitwise-identical trajectory.

Result scope. The current names Equilibrium and Complement correspond to the former Static and Zero formulas, respectively. Renaming does not change those formulas. The no-RA Small run retained its original Static label in its training record; its shared-frame core and visual shell match the model reported here. Earlier LSSO/Ridgon variants used different generators, projections, positional mechanisms, or training protocols. Their ImageNet, DNA, LRA, and operator-timing measurements are not reported as measurements of the current model. Base training is ongoing and has no final result in this draft.

Figures and writing tools. The overview is a vector schematic of the stated equations. Validation plots use all recorded epochs without smoothing or interpolated missing values. The accompanying plotting script reconstructs figures from local run records and records their provenance. Scientific Agent Skills [9] provided writing and figure-review procedures for this revision; the draft remains subject to author verification.

References

[1] Arshia Afzal, Elias Abad Rocamora, Leyla Naz Candogan, Pol Puigdemont, Francesco Tonin, Yongtao Wu, Mahsa Shoaran, and Volkan Cevher. Linear attention for efficient bidirectional sequence modeling. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. CMS Books in Mathematics. Springer, Cham, 2 edition, 2017.

[4] Xiangxiang Chu, Zhi Tian, Bo Zhang, Xinlong Wang, and Chunhua Shen. Conditional positional encodings for vision transformers. In *International Conference on Learning Representations*, 2023.

[5] Marta Garnelo and Wojciech Marian Czarnecki. Exploring the space of key-value-query models with intention. *arXiv preprint arXiv:2305.10203*, 2023.

[6] Dongchen Han, Yining Li, Tianyu Li, Zixuan Cao, Ziming Wang, Jun Song, Yu Cheng, Bo Zheng, and Gao Huang. ViT^3: Unlocking test-time training in vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 51–61, 2026.

[7] Greg Henry, Ping Tak Peter Tang, and Alexander Heinecke. Leveraging the bfloat16 artificial intelligence datatype for higher-precision computations. *arXiv preprint arXiv:1904.06376*, 2019.

[8] Sukjun Hwang, Aakash Lahoti, Ratish Puduppully, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[9] Timothy Kassis, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. *arXiv preprint arXiv:2609.00065*, 2026.

[10] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.

[11] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115:211–252, 2015.

[12] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.

[13] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to Learn at test time: RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2025.

[14] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers and distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021.

[15] Johannes von Oswald, Nino Scherrer, Seijin Kobayashi, Luca Versari, Songlin Yang, Sarthak Mittal, Maximilian Schlegel, Kaitlin Maile, Yanick Schimpf, Oliver Sieberling, Alexander Meulemans, Rif A. Saurous, Guillaume Lajoie, Charlotte Frenkel, Razvan Pascanu, Blaise Agüera y Arcas, and João Sacramento. MesaNet: Sequence modeling by locally optimal test-time training. In *International Conference on Learning Representations*, 2026.

[16] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: A unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.

[17] Ezra Winston and J. Zico Kolter. Monotone operator equilibrium networks. In *Advances in Neural Information Processing Systems*, 2020.

[18] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2025.

[19] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *International Conference on Machine Learning*, pages 62429–62442, 2024.